

Final Project – Cloud Computing and Virtualization v2

MEI-IoT – Masters in Informatics Engineering: Internet of Things

Prof. Renato Panda (renato.panda@ipt.pt)

2026-05-13

Table of contents

1	Final Project – Cloud Computing and Virtualization	2
1.1	Introduction	2
1.2	Learning Outcomes	2
1.3	Objectives	2
1.4	Project Context	2
1.5	Group Work	3
1.6	Project Tasks	3
1.6.1	A. Virtual Machines (On-Premises IaaS Simulation)	3
1.6.2	B. Containers & Cloud-Native (On-Premises or Public Cloud)	4
1.7	Intermediate Checkpoints and Technical Discussions	4
1.7.1	Checkpoint 1 — Original System Analysis and Initial Plan	4
1.7.2	Checkpoint 2 — Solution A Architecture and Prototype	5
1.7.3	Checkpoint 3 — Solution A Consolidation and Solution B Plan	5
1.7.4	Checkpoint 4 — Solution B Prototype and Comparative Analysis	5
1.7.5	Use of AI and External Assistance	6
1.7.6	Decision Log	6
1.8	Deliverables	6
1.9	Submission Guidelines	7
1.10	Suggestions and Tips	7
1.11	Resources	8
1.12	Assessment Criteria	8
1.13	Deadline	9

1 Final Project – Cloud Computing and Virtualization

1.1 Introduction

The project is designed to consolidate your understanding of modern cloud and virtualization technologies, focusing on the design, deployment, and critical analysis of scalable and resilient infrastructures and applications.

1.2 Learning Outcomes

Upon successful completion, you will be able to:

- Analyze and critique application architectures for scalability and resilience.
- Design and implement distributed systems using VMs and containers.
- Apply cloud-native and automation tools to real-world scenarios.
- Justify architectural decisions and reflect on trade-offs.

1.3 Objectives

By completing this project, you will:

- Develop practical skills in:
 - Provisioning tools
 - Load balancing
 - High availability
 - Service discovery
 - Containers and cloud-native technologies
 - Scalability and elasticity
- Learn to critically analyze and improve real-world application architectures.
- Demonstrate the ability to justify design decisions and reflect on trade-offs.

1.4 Project Context

Modern infrastructures must be resilient and capable of horizontal scaling to fully leverage virtualization and cloud computing. In this project, you will start with **Project O** — a simple PHP web application provided as a baseline Vagrant setup — and reimagine its deployment so that it becomes **scalable, resilient, and highly available**.

You will explore two distinct approaches (A and B), each emphasizing different aspects of cloud, containerization, and virtualization.

Project O (the original application) includes:

- Static content
- Data loading from external APIs (e.g., cat pictures)
- JavaScript libraries
- Sessions
- File uploads
- Relational database
- WebSocket communication

1.5 Group Work

- **Default:** The project should be done in groups of 2 students, but both students must actively participate to ensure hands-on experience with all tools and concepts.
- **Exception:** Individual work is allowed if justified (e.g., scheduling conflicts, other commitments). Please contact the instructor in advance to discuss and approve individual work.

1.6 Project Tasks

You will design, implement, and analyze two alternative architectures for the provided web application:

1.6.1 A. Virtual Machines (On-Premises IaaS Simulation)

- Use **Vagrant** and virtual machines to deploy the application in a more robust, distributed, and scalable manner, simulating an on-premises IaaS environment.
- Use **Ubuntu** or **Alpine Linux** (recommended to save resources) as the VM operating system.
- You are free to explore and combine any of the tools and techniques covered in class, including (but not limited to):
 - **Automation:** Ansible (provisioning, configuration management)
 - **Load balancing:** nginx (or similar)
 - **High availability:** Keepalived (VRRP failover for load balancers)
 - **Database HA:** repmgr (PostgreSQL streaming replication), Patroni (automatic PostgreSQL failover)
 - **Sessions:** centralized session storage (e.g., Redis, Memcached)
 - **Storage:** shared/distributed storage for file uploads (e.g., GlusterFS, NFS)
 - **Service discovery:** Consul (automatic registration and load balancer updates)
 - **Monitoring & Observability:** Prometheus, Grafana, Loki, SigNoz

1.6.2 B. Containers & Cloud-Native (On-Premises or Public Cloud)

- Use **Docker containers** and orchestration (Docker Swarm, Kubernetes, or similar) to deploy the application, either on-premises or using a public cloud provider.
- Students will have access to **Google Cloud Platform (GCP) education credits** to explore managed cloud services (Compute Engine, Cloud SQL, Cloud Storage, GKE, Cloud Load Balancing, etc.).
- You may also explore **GCP emulators** locally before deploying to the cloud:
 - [gcloud emulators](#)
 - [gcloud beta emulators](#)
 - [fake-gcs-server](#) (GCS emulator)
 - [fullstorydev/emulators](#) (Pub/Sub, Bigtable, etc.)
- Integrate IaaS or cloud services as appropriate for your design. The same topics listed for Option A are valid here, applied in a container/cloud context.

For both approaches, you are expected to:

- Critically analyze **Project O** and its original architecture.
- Propose and implement an improved, scalable, resilient, and highly available architecture.
- Justify your design choices and discuss any limitations or trade-offs.

1.7 Intermediate Checkpoints and Technical Discussions

During the project, you must complete several intermediate checkpoints. Each checkpoint requires a short Moodle submission, preferably **3–4 slides**, to support a brief **10-minute technical discussion** with the practical class instructor.

The goal is not to create polished mini-presentations. The checkpoints are intended to show progress, justify decisions, identify difficulties, and demonstrate that you understand the system you are building. The exact timing of each discussion may be adjusted or agreed with the practical class instructor.

1.7.1 Checkpoint 1 — Original System Analysis and Initial Plan

Suggested date: 21/05/2026

Present:

- A simple architecture diagram of Project O.
- Main limitations of the original application regarding scalability, resilience, high availability, sessions, file uploads, database, and WebSockets.
- Initial requirements for improving the system.
- Initial plan for Solution A using Vagrant/VMs.

1.7.2 Checkpoint 2 — Solution A Architecture and Prototype

Suggested date: 04/06/2026

Present:

- Planned architecture for Solution A.
- Current implementation status.
- Tools selected and why they were selected.
- Evidence of progress, such as commands, screenshots, configuration snippets, logs, or a short demo.
- Main technical risks and next steps.

1.7.3 Checkpoint 3 — Solution A Consolidation and Solution B Plan

Suggested date: 18/06/2026

Present:

- Implemented architecture for Solution A.
- Differences between planned and implemented architecture.
- Evidence of testing, such as load balancing, failure simulation, persistence, monitoring, or performance tests.
- Limitations of Solution A.
- Initial architecture and rationale for Solution B using containers, orchestration, cloud services, or a hybrid approach.

1.7.4 Checkpoint 4 — Solution B Prototype and Comparative Analysis

Suggested date: 25/06/2026

Present:

- Implemented or nearly implemented architecture for Solution B.
- Current technical status and remaining work.
- Comparison between Project O, Solution A, and Solution B.
- Final plan for the report, demo, and defense.

1.7.5 Use of AI and External Assistance

You may use AI-based tools and external resources for learning, debugging, documentation, and exploration. However, you remain fully responsible for understanding, validating, and explaining all submitted work.

The final report should include a short note describing which AI tools or external assistance were used, for what purpose, and how the outputs were validated.

1.7.6 Decision Log

The final report should include a brief decision log summarizing relevant architectural and technical decisions. For each major decision, briefly mention:

- The problem or requirement.
- Alternatives considered.
- Chosen solution.
- Reason for the choice.
- Limitations or trade-offs.

Examples of useful decisions to document:

- Why use Redis, Memcached, the SQL database or XPTO for sessions?
- Why use NFS, GlusterFS, or object storage for uploads?
- Why use nginx, HAProxy, or a cloud load balancer?
- Why use Docker Compose, Docker Swarm, Kubernetes, GCP, or a hybrid approach?
- Why implement or not implement database high availability?

1.8 Deliverables

Submit a **ZIP** file containing all relevant materials:

- **Project Report** (recommended structure):
 1. **Introduction and Objectives**
 2. **Analysis of the Original Application**
 - System architecture diagram
 - Technologies used
 - Identified issues and limitations
 - (Optional) Performance metrics (e.g., using **hey** or **Vegeta**)
 3. **Proposed Solutions**
 - Separate sections for A (VMs) and B (Containers/Cloud)
 - Planned/ideal vs. implemented architecture (with architecture diagrams)

- Main components, rationale, and implementation details
- (Optional) Performance metrics
- 4. **Engineering Process and Decision Log**
 - Main decisions, alternatives considered, and trade-offs
 - Evidence of testing, debugging, and iteration
 - Short note on the use of AI tools or external assistance
- 5. **Comparison and Conclusions**
 - Comparison between Project O, Solution A, and Solution B
 - Key takeaways and lessons learned
- **Configuration/code and instructions** to replicate/run your solution.
- **Slides and demo** (max 15 minutes; slides may be submitted after the report).

1.9 Submission Guidelines

- Submit your ZIP file via the platform or as instructed by the professor.
- Ensure all files are clearly named and organized.
- Include a README with student name(s), instructions, and any special notes.

1.10 Suggestions and Tips

- **Critical Thinking:**

Discuss features you considered, even if not implemented. Explain your reasoning and trade-offs.
- **Potential Features to Explore:**
 - Three-layer architecture (load balancer, web, database)
 - Serving the website correctly (partially or fully)
 - Manual or semi-automatic scaling of instances
 - Automatic service discovery and dynamic load balancer updates (e.g., Consul)
 - Auto-scaling the web layer (e.g., based on schedule or CPU load)
 - High availability for the load balancer (e.g., Keepalived / VRRP, cloud LB)
 - High availability for the database (e.g., repmgr, Patroni, managed cloud DB)
 - Centralized session storage to support horizontal web scaling (e.g., Redis)
 - Shared/distributed storage for file uploads (e.g., GlusterFS, NFS, cloud object storage)
 - Monitoring & observability stack (e.g., Prometheus + Grafana, Loki, SigNoz)
 - Automated deployment and configuration management (e.g., Ansible, Terraform/OpenTofu)
- **Documentation:**

Use diagrams, tables, and clear explanations. Reference all external resources and tools used.

1.11 Resources

- **Project O repository:** https://github.com/renatopanda/ipt_cloud_course_project (*link may be updated — check the course platform for the latest version*)
- **Recommended tools:** Vagrant, Docker, Google Cloud Platform, and tools explored during classes: Ansible, nginx, Consul, Keepalived, repmgr, Patroni, Redis, GlusterFS, Prometheus, Grafana, Loki, SigNoz, Docker Swarm, Kubernetes, Terraform/OpenTofu, hey, Vegeta, etc.
- **GCP Emulators (Option B — local development):**
 - [gcloud emulators](#)
 - [gcloud beta emulators](#)
 - [fake-gcs-server](#) (GCS emulator)
 - [fullstorydev/emulators](#) (Pub/Sub, Bigtable, etc.)

1.12 Assessment Criteria

Your project will be evaluated based on the following **objective criteria**:

- **Analysis and Planning (15%)**
 - Depth and clarity of the original system analysis
 - Identification of relevant issues and requirements
- **Design and Implementation (35%)**
 - Appropriateness and justification of architectural choices
 - Correctness and completeness of the implemented solutions (A & B)
 - Use of relevant tools and technologies
- **Documentation and Communication (15%)**
 - Quality, clarity, and organization of the report and diagrams
 - Clear, reproducible instructions and code/configuration
- **Critical Reflection (15%)**
 - Discussion of limitations, trade-offs, and lessons learned
 - Comparison between original and proposed solutions
- **Intermediate Checkpoints and Technical Discussion (10%)**
 - Evidence of steady progress and response to feedback
 - Ability to explain decisions, risks, and implementation status
- **Final Presentation and Defense (10%)**
 - Clarity and effectiveness of the oral presentation/demo

- Ability to answer questions and demonstrate understanding

All students must attend the project defense and demonstrate a clear understanding of the solution.

1.13 Deadline

- **Final submission:** around **01/07/2026**, to be confirmed on the course platform.
- Intermediate checkpoint dates are suggested dates and may be adjusted by the practical class instructor.

For reference and updates, please consult the project repository and the course platform.